# Atelier Machine Learning

*Analyse Comportementale Clientèle Retail*

## COMPTE RENDU

Atelier Pratique : E-commerce de Cadeaux

**Préparé par Mariem Ben Abdallah**

---

**Encadrante : Fadoua Drira**

**Année Universitaire : 2025-2026**

# Table des matières

**1 Introduction et Contexte** **3**
1.1 Présentation du projet . . . 3
1.2 Objectifs métier . . . 3
1.3 Objectifs pédagogiques . . . 3
1.4 Structure du rapport . . . 3

**2 Exploration et Analyse des Données** **4**
2.1 Présentation du dataset . . . 4
2.2 Analyse des types de données . . . 4
2.3 Valeurs manquantes . . . 4
2.4 Valeurs aberrantes et codes spéciaux . . . 4
2.5 Feature Engineering réalisé . . . 5
2.6 Visualisations exploratoires . . . 5

**3 Préparation des Données** **6**
3.1 Nettoyage des données . . . 6
3.2 Suppression des features leakantes — Point critique . . . 6
3.3 Gestion de la multicolinéarité . . . 6
3.4 Split Train/Test . . . 6
3.5 Imputation des valeurs manquantes — Correction critique . . . 7
3.6 Encodage des variables catégorielles . . . 7
3.7 Normalisation des données . . . 7

**4 Modélisation** **8**
4.1 Stratégie de modélisation . . . 8
4.2 Modèle 1 : KNN (K-Nearest Neighbors) . . . 8
4.2.1 Principe . . . 8
4.2.2 Optimisation réalisée . . . 8
4.2.3 Résultats KNN . . . 8
4.3 Modèle 2 : Arbre de Décision . . . 8
4.3.1 Principe . . . 8
4.3.2 Optimisation réalisée . . . 9
4.3.3 Résultats Arbre de Décision . . . 9
4.4 Modèle 3 : Random Forest . . . 9
4.4.1 Principe . . . 9
4.4.2 Paramètres utilisés . . . 9
4.4.3 Résultats Random Forest . . . 9
4.5 Modèle 4 : Gradient Boosting — Meilleur modèle . . . 9
4.5.1 Principe . . . 9
4.5.2 Optimisation par GridSearchCV . . . 10
4.5.3 Résultats Gradient Boosting . . . 10
4.6 Top features importantes . . . 10
4.7 Tableau comparatif final . . . 11
4.8 Choix du modèle final . . . 11

**5 Segmentation Clientèle (ACP + K-means)** **12**
5.1 Objectif de la segmentation . . . 12
5.2 Prétraitement pour la segmentation . . . 12
5.3 Analyse en Composantes Principales (ACP) . . . 12
5.3.1 Principe . . . 12
5.3.2 Résultats de l'ACP . . . 12

5.4 Détermination du nombre de clusters (K-means) . . . 12
5.4.1 Principe de K-means . . . 12
5.4.2 Méthode du coude . . . 12
5.5 Résultats de la segmentation . . . 13
5.6 Interprétation des clusters . . . 13
5.6.1 Cluster 0 — Clients Réguliers (Risque faible) . . . 13
5.6.2 Cluster 1 — Occasionnels à Risque modéré . . . 13
5.6.3 Cluster 2 — Occasionnels à Risque élevé . . . 13
5.6.4 Cluster 3 — Clients Réguliers (Risque élevé) . . . 13
5.6.5 Outliers — Cas atypiques . . . 13

**6 Déploiement avec Flask** **14**
6.1 Choix technologique . . . 14
6.2 Architecture de l'application . . . 14
6.3 API REST implémentée . . . 14
6.4 Fonctionnalités implémentées . . . 14
6.4.1 Interface utilisateur . . . 14
6.4.2 Prétraitement en temps réel . . . 14
6.5 Exemples de prédictions . . . 15
6.6 Logs et monitoring . . . 15
6.7 Utilisation de l'API REST . . . 15

**7 Conclusion et Perspectives** **16**
7.1 Résumé des réalisations . . . 16
7.2 Points forts du projet . . . 16
7.3 Limites identifiées . . . 16
7.4 Perspectives d'amélioration . . . 16
7.5 Livrables du projet . . . 16

**Annexes** **17**

# 1 Introduction et Contexte

## 1.1 Présentation du projet

Ce projet s'inscrit dans le cadre de l'atelier pratique du module Machine Learning. L'objectif est de réaliser une **analyse comportementale complète** de la clientèle d'une entreprise e-commerce spécialisée dans la vente de cadeaux.

Le projet suit une méthodologie rigoureuse couvrant l'ensemble de la chaîne de traitement en data science : de l'exploration des données brutes jusqu'au déploiement d'une application web interactive.

## 1.2 Objectifs métier

L'entreprise e-commerce souhaite mieux comprendre sa clientèle pour :

- **Personnaliser** ses stratégies marketing (offres ciblées, communications adaptées)
- **Réduire** le taux de départ des clients (churn) en anticipant les départs
- **Optimiser** son chiffre d'affaires en fidélisant les clients à forte valeur
- **Anticiper** les comportements à risque pour agir avant le départ

## 1.3 Objectifs pédagogiques

**Compétences développées**

Ce projet couvre l'ensemble de la chaîne de traitement en data science :

- Exploration des données
- Préparation et nettoyage
- Feature engineering
- Transformation (ACP)
- Modélisation (Clustering, Classification)
- Évaluation et interprétation
- Optimisation des hyperparamètres
- Déploiement (Flask)

## 1.4 Structure du rapport

Ce rapport est organisé en plusieurs sections :

1. **Exploration des données** : analyse statistique, valeurs manquantes, visualisations
2. **Préparation des données** : nettoyage, feature engineering, encodage
3. **Modélisation** : entraînement et comparaison de 4 modèles
4. **Segmentation clientèle** : ACP et K-means pour identifier des profils
5. **Déploiement** : application Flask et API REST
6. **Conclusion** : bilan des résultats et perspectives

# 2 Exploration et Analyse des Données

## 2.1 Présentation du dataset

Le jeu de données fourni contient **4 372 clients** et **52 features**. Il s'agit de données intentionnellement imparfaites (valeurs manquantes, outliers, features redondantes) pour nous permettre de maîtriser l'ensemble des étapes de prétraitement.

**Structure des données**

| Type de features | Nombre | Exemples |
|---|---|---|
| Numériques | 34 | Recency, Frequency, Monetary, Age |
| Catégorielles | 18 | RFMSegment, CustomerType, Region |

## 2.2 Analyse des types de données

Nous avons commencé par analyser la structure du dataset :

- **Features numériques** : variables quantitatives (RFM, quantités, ratios)
- **Features catégorielles** : segments clients, préférences, statuts
- **Features temporelles** : dates d'inscription, dernières connexions
- **Variable cible** : `Churn` (0 = fidèle, 1 = partant)

## 2.3 Valeurs manquantes

L'analyse des valeurs manquantes a révélé :

| Colonne | Manquantes | Pourcentage |
|---|---|---|
| Age | 1 311 | 29.99% |
| AvgDaysBetweenPurchases | 79 | 1.81% |

Table 1 – Valeurs manquantes par colonne

**Interprétation :**

- Les 79 clients sans délai entre achats sont ceux qui n'ont effectué **qu'un seul achat** (Frequency=1)
- L'âge manque pour environ 30% des clients → nécessite une imputation

## 2.4 Valeurs aberrantes et codes spéciaux

Certaines colonnes contiennent des valeurs spéciales qui ne sont pas des erreurs mais des codes métier :

- **SupportTicketsCount** : valeurs -1 (code pour "non applicable") et 999 (code pour "trop élevé")
- **SatisfactionScore** : valeurs -1, 0, 99 (codes spéciaux pour "non renseigné")
- **MonetaryTotal négatif** : 44 clients avec dépense totale négative (retours/annulations)

### 2.5 Feature Engineering réalisé

Pour enrichir notre jeu de données, nous avons créé de nouvelles features :

**Nouvelles features créées**

- `HasNegativeMonetary` : flag binaire indiquant si le client a une dépense négative
- `RegYear, RegMonth, RegDay, RegWeekday` : extraction depuis la date d'inscription
- `IP_IsPrivate` : indique si l'adresse IP est privée (1) ou publique (0)
- `IP_Class` : classe de l'adresse IP (A, B, C, Other)
- `IP_FirstOctet` : premier octet de l'adresse IP

### 2.6 Visualisations exploratoires

Plusieurs visualisations ont été réalisées pour comprendre la distribution des données :

- Distribution du MonetaryTotal (dépense totale)
- Corrélation entre les features numériques (heatmap)
- Taux de churn par mois d'inscription
- Répartition des clients par catégorie d'âge et de dépense

# 3 Préparation des Données

## 3.1 Nettoyage des données

Le nettoyage a consisté à :

- **Supprimer les colonnes constantes** : NewsletterSubscribed = "Yes" pour 100% des clients
- **Parser les dates** : conversion de RegistrationDate en format datetime
- **Corriger les types** : s'assurer que chaque colonne a le bon type
- **Supprimer les identifiants** : CustomerID retiré (non prédictif)

## 3.2 Suppression des features leakantes — Point critique

Une attention particulière a été portée à l'identification et la suppression des features causant du **data leakage**. Le data leakage se produit lorsqu'une feature utilise une information qui ne serait pas disponible au moment de la prédiction.

**Features leakantes supprimées**

- `ChurnRiskCategory, RFMSegment, CustomerType, LoyaltyLevel` → construites APRÈS avoir défini le churn (leakage direct)
- `FirstPurchaseDaysAgo, CustomerTenureDays` → calculées sur la même période que le label (leakage temporel)
- `Recency` → corrélation de 0.859 avec le churn (donnerait une précision artificielle de 100%)
- `PreferredMonth, AvgDaysBetweenPurchases` → dérivées des mêmes informations temporelles

## 3.3 Gestion de la multicolinéarité

La multicolinéarité désigne une forte corrélation entre plusieurs variables explicatives. Cela peut poser problème pour certains modèles (régression linéaire, logistique).

**Méthode utilisée :**

1. Calcul de la matrice de corrélation
2. Identification des paires avec $|\text{corrélation}| > 0.8$
3. Suppression d'une feature par paire redondante

| Feature 1 (conservée) | Feature 2 (supprimée) | Corrélation |
|---|---|---|
| Frequency | UniqueInvoices | 1.000 |
| UniqueProducts | UniqueDescriptions | 0.999 |
| MonetaryMin | MonetaryMax | -0.994 |
| MonetaryTotal | TotalQuantity | 0.922 |

Table 2 – Paires de features fortement corrélées

**Résultat :** 7 features redondantes supprimées.

## 3.4 Split Train/Test

Le split des données est une étape cruciale pour évaluer correctement les performances d'un modèle sur des données jamais vues.

**Split réalisé**

- Proportion : 80% train / 20% test
- Méthode : train_test_split avec random_state=42 (reproductibilité)
- Stratification : sur la variable Churn (préserve la proportion des classes)
- Résultat : 3 497 clients en train, 875 en test

### 3.5 Imputation des valeurs manquantes — Correction critique

L'imputation des valeurs manquantes doit être effectuée **APRÈS** le split pour éviter tout data leakage.

**Bonne pratique respectée**

1. Split Train/Test
2. Identification des colonnes avec NaN dans X_train
3. SimpleImputer fit sur X_train uniquement (stratégie = médiane)
4. Transformation de X_train et X_test avec le même imputer

**Colonne imputée :** Age (1311 valeurs manquantes) → remplacées par la médiane.

### 3.6 Encodage des variables catégorielles

Les modèles de machine learning ne peuvent pas traiter directement les variables textuelles. Nous avons donc appliqué un **One-Hot Encoding**.

- **12 variables catégorielles** encodées
- **drop_first=True** pour éviter la multicolinéarité
- Nombre total de features après encodage : **103**

**Attention particulière :** Certaines catégories présentes uniquement dans le train (ex : Country_Brazil, Region_Amérique du Nord) ont été remplies avec 0 dans le test.

### 3.7 Normalisation des données

Pour les modèles sensibles aux échelles (KNN, SVM), nous avons normalisé les données avec **StandardScaler**.

**Normalisation après split**

- StandardScaler fit sur X_train_encoded uniquement
- Transformation de X_train_encoded et X_test_encoded
- Sauvegarde du scaler pour l'application Flask

# 4 Modélisation

## 4.1 Stratégie de modélisation

Nous avons choisi de tester **4 modèles** aux architectures différentes pour comparer leurs performances et sélectionner le meilleur :

1. **KNN** : modèle non paramétrique basé sur la proximité
2. **Arbre de Décision** : modèle interprétable
3. **Random Forest** : ensemble d'arbres (bagging)
4. **Gradient Boosting** : ensemble séquentiel (boosting)

## 4.2 Modèle 1 : KNN (K-Nearest Neighbors)

### 4.2.1 Principe

Le KNN classe un nouveau client en fonction des k clients les plus proches dans l'espace des features. C'est un modèle non paramétrique qui ne fait pas d'hypothèse sur la distribution des données.

### 4.2.2 Optimisation réalisée

Le KNN a bénéficié d'une optimisation poussée avec 3 stratégies :

1. **SelectKBest** : sélection des 20 meilleures features (test de Fisher)
2. **PCA** : réduction de dimension (testée mais moins performante)
3. **SelectKBest + PCA** : combinaison des deux approches

**Paramètres optimaux trouvés :**

— Nombre de voisins (k) : 15

— Poids : distance (les voisins proches ont plus d'influence)

— Métrique : Manhattan

— Features sélectionnées : 20 meilleures

### 4.2.3 Résultats KNN

| Métrique | KNN Base | KNN Optimisé | Gain |
|---|---|---|---|
| Accuracy | 0.753 | 0.878 | +12.5% |
| Recall | 0.430 | 0.825 | +39.5% |
| F1-Score | 0.536 | 0.818 | +28.1% |
| ROC-AUC | 0.789 | 0.925 | +13.6% |

Table 3 – Amélioration du KNN après optimisation

## 4.3 Modèle 2 : Arbre de Décision

### 4.3.1 Principe

L'arbre de décision segmente l'espace des features en régions homogènes selon la variable cible. Il est très interprétable mais sujet au sur-apprentissage.

### 4.3.2 Optimisation réalisée

- Critère : entropie
- Contrainte : max_depth=5 (pour éviter le sur-apprentissage)
- Class_weight = 'balanced' (gestion du déséquilibre)
- Min_samples_split = 20, min_samples_leaf = 10

**Impact de la profondeur maximale**

| | Sans contrainte | max_depth=5 |
|---|---|---|
| Profondeur | 18 | 5 |
| Accuracy Train | 1.000 | 0.874 |
| Accuracy Test | 0.839 | 0.864 |
| Écart | 16.1% | 1.0% |

### 4.3.3 Résultats Arbre de Décision

- **F1-Score** : 0.819
- **Recall** : 0.924 (le meilleur parmi tous les modèles !)
- **ROC-AUC** : 0.944

L'arbre de décision détecte **92.4% des clients partants** — excellent pour une stratégie de fidélisation préventive.

## 4.4 Modèle 3 : Random Forest

### 4.4.1 Principe

La Random Forest construit de nombreux arbres de décision sur des échantillons bootstrappés et agrège leurs prédictions. Cela réduit la variance et améliore la généralisation.

### 4.4.2 Paramètres utilisés

- n_estimators = 200 arbres
- max_depth = 10
- min_samples_split = 10
- max_features = 'sqrt'
- class_weight = 'balanced'
- oob_score = True (validation out-of-bag)

### 4.4.3 Résultats Random Forest

- **F1-Score** : 0.783
- **ROC-AUC** : 0.907
- **OOB Score** : 0.845

## 4.5 Modèle 4 : Gradient Boosting — Meilleur modèle

### 4.5.1 Principe

Le Gradient Boosting construit les arbres séquentiellement : chaque nouvel arbre corrige les erreurs du précédent. C'est souvent le modèle le plus performant sur les données tabulaires.

### 4.5.2 Optimisation par GridSearchCV

Nous avons effectué une recherche d'hyperparamètres sur 3 dimensions :

| Paramètre | Valeurs testées | Meilleure valeur |
|---|---|---|
| n_estimators | 100, 200 | 200 |
| learning_rate | 0.01, 0.1, 0.2 | 0.01 |
| max_depth | 3, 5, 7 | 5 |

Table 4 – GridSearchCV - Gradient Boosting

**Validation croisée :** F1 moyen = 0.827 (5 folds)

### 4.5.3 Résultats Gradient Boosting

**Modèle final retenu**

- **F1-Score** : **0.8297**
- **ROC-AUC** : **0.9471**
- **Accuracy** : 89.3%
- **Precision** : 0.877
- **Recall** : 0.787

## 4.6 Top features importantes

L'analyse des importances des features du Gradient Boosting révèle :

| Feature | Importance | Interprétation |
|---|---|---|
| FavoriteSeason_Hiver | 22.4% | Saison d'achat préférée |
| FavoriteSeason_Été | 20.0% | Saison d'achat préférée |
| Frequency | 19.6% | Fréquence d'achat |
| FavoriteSeason_Printemps | 14.2% | Saison d'achat préférée |
| TotalTransactions | 9.0% | Nombre total de transactions |
| MonetaryTotal | 3.3% | Dépense totale |

Table 5 – Top 6 features importantes

**Analyse des features importantes**

**Observation :** Les features FavoriteSeason (Hiver, Été, Printemps) représentent 56.5% de l'importance totale du modèle.
**Explication possible :** Les données couvrent une période spécifique (2009-2011) où la saison d'achat est corrélée avec la période de labellisation du churn. Ce n'est pas nécessairement du leakage mais une corrélation spurieuse à mentionner dans les limites du projet.

## 4.7 Tableau comparatif final

| Modèle | Accuracy | Recall | F1-Score | ROC-AUC |
|---|---|---|---|---|
| **Gradient Boosting** | **0.8926** | 0.7869 | **0.8297** | **0.9471** |
| Arbre Décision | 0.8640 | **0.9244** | 0.8189 | 0.9440 |
| KNN Optimisé | 0.8777 | 0.8247 | 0.8177 | 0.9249 |
| Random Forest | 0.8549 | 0.7869 | 0.7829 | 0.9071 |

Table 6 – Comparaison des performances des modèles

## 4.8 Choix du modèle final

Nous retenons le **Gradient Boosting** comme modèle final pour les raisons suivantes :

1. **Meilleur F1-Score** (0.83) : équilibre optimal entre précision et rappel
2. **Meilleur ROC-AUC** (0.947) : excellent pouvoir discriminant
3. **Meilleure Accuracy** (89.3%) : plus de prédictions correctes
4. **Modèle robuste** : bien adapté aux données tabulaires complexes

# 5 Segmentation Clientèle (ACP + K-means)

## 5.1 Objectif de la segmentation

La segmentation clientèle vise à identifier des groupes homogènes de clients pour adapter les stratégies marketing. Nous utilisons une approche non supervisée (ACP + K-means).

## 5.2 Prétraitement pour la segmentation

1. **Sélection des features numériques** : 27 colonnes conservées
2. **Imputation** : médiane pour les valeurs manquantes
3. **Normalisation** : StandardScaler (indispensable pour K-means)
4. **Retrait des outliers** : Z-score $> 3$ sur au moins une dimension

**Outliers détectés**

— **Outliers identifiés** : 817 clients (23.4% du dataset)
— **Clients conservés** : 2 680 clients (76.6%)
— **Profil des outliers** : fréquence moyenne 7.7 achats, dépense moyenne 3842€

## 5.3 Analyse en Composantes Principales (ACP)

### 5.3.1 Principe

L'ACP est une technique de réduction de dimension qui transforme les variables corrélées en variables indépendantes (composantes principales) tout en conservant le maximum de variance.

### 5.3.2 Résultats de l'ACP

— **14 composantes** retenues pour expliquer 80% de la variance
— **20 composantes** nécessaires pour 95% de la variance
— Visualisation 2D possible pour l'interprétation

## 5.4 Détermination du nombre de clusters (K-means)

### 5.4.1 Principe de K-means

K-means partitionne les données en k clusters en minimisant la distance intra-cluster. Chaque cluster est représenté par son centroïde.

### 5.4.2 Méthode du coude

Nous avons utilisé la méthode du coude (Elbow method) qui consiste à tracer l'inertie en fonction du nombre de clusters k. Le "coude" indique le k optimal.

— Test de k=2 à 10
— Coude détecté autour de k=3 ou k=4
— Analyse des tailles minimales : k=4 retenu (plus petit cluster = 222 clients, 8.3%)

## 5.5 Résultats de la segmentation

| Cluster | Nom du segment | Taille | Churn | Profil |
|---|---|---|---|---|
| 0 | Clients Réguliers | 222 (8.3%) | 19.4% | 6.1 achats, 1735£ |
| 1 | Occasionnels à Risque | 744 (27.8%) | 29.7% | 3.5 achats, 1121£ |
| 2 | Occasionnels à Risque | 747 (27.9%) | 34.3% | 2.9 achats, 902£ |
| 3 | Clients Réguliers | 967 (36.1%) | 37.6% | 5.7 achats, 1697£ |
| - | Outliers | 817 (23.4%) | 34.1% | Analyse individuelle |

Table 7 – Récapitulatif des segments clients

## 5.6 Interprétation des clusters

### 5.6.1 Cluster 0 — Clients Réguliers (Risque faible)

- **Taille** : 222 clients (8.3%)
- **Taux de churn** : 19.4% (le plus bas)
- **Profil** : fréquence d'achat élevée (6.1), dépense importante (1735£)
- **Action recommandée** : programme de fidélité, upselling, cross-selling

### 5.6.2 Cluster 1 — Occasionnels à Risque modéré

- **Taille** : 744 clients (27.8%)
- **Taux de churn** : 29.7%
- **Profil** : fréquence moyenne (3.5), dépense modérée (1121£)
- **Action recommandée** : newsletter ciblée, promotions personnalisées

### 5.6.3 Cluster 2 — Occasionnels à Risque élevé

- **Taille** : 747 clients (27.9%)
- **Taux de churn** : 34.3%
- **Profil** : fréquence faible (2.9), dépense faible (902£)
- **Action recommandée** : offres de réactivation urgentes

### 5.6.4 Cluster 3 — Clients Réguliers (Risque élevé)

- **Taille** : 967 clients (36.1%)
- **Taux de churn** : 37.6% (le plus élevé des clusters)
- **Profil** : fréquence élevée (5.7) mais churn élevé (paradoxe)
- **Action recommandée** : campagne anti-churn prioritaire

### 5.6.5 Outliers — Cas atypiques

- **Taille** : 817 clients (23.4%)
- **Profil** : comportements extrêmes sur au moins une dimension
- **Action recommandée** : analyse individuelle (pas de campagne marketing générique)

# 6 Déploiement avec Flask

## 6.1 Choix technologique

Flask a été choisi pour le déploiement car c'est un micro-framework Python :

- Léger et flexible
- Facile à intégrer avec Scikit-learn
- Idéal pour créer des API REST
- Parfait pour un projet académique

## 6.2 Architecture de l'application

L'application Flask se compose de :

1. **app.py** : le serveur backend
2. **templates/index.html** : l'interface utilisateur
3. **Modèles sauvegardés** : best_model_churn.pkl, scaler.pkl

## 6.3 API REST implémentée

**Endpoints disponibles**

| Endpoint | Fonction |
|---|---|
| GET / | Interface web |
| POST /predict | Prédiction via formulaire |
| POST /api/predict | Prédiction via API JSON |
| GET /health | Vérification de santé |
| GET /debug | Informations de debug |

## 6.4 Fonctionnalités implémentées

### 6.4.1 Interface utilisateur

- Formulaire avec 6 champs (Recency, Frequency, Monetary, Age, Satisfaction, Tickets)
- Boutons d'exemples pré-remplis (client fidèle / client à risque)
- Jauge de risque visuelle (couleurs vert/orange/rouge)
- Scores RFM calculés automatiquement
- Conseils personnalisés selon le niveau de risque
- Historique des 10 dernières prédictions

### 6.4.2 Prétraitement en temps réel

L'application reconstruit un vecteur de features complet (103 dimensions) à partir des données saisies :

- Valeurs numériques saisies par l'utilisateur
- Valeurs par défaut pour les features non renseignées
- Encodage one-hot des variables catégorielles
- Normalisation avec le scaler sauvegardé

## 6.5 Exemples de prédictions

| Profil | Freq | Monetary | Sat | Risque | Pred |
|---|---|---|---|---|---|
| Client Premium | 50 | 10000€ | 5 | 15% | Fidèle |
| Client Moyen | 5 | 500€ | 4 | 40% | Fidèle |
| Client Critique | 1 | 50€ | 1 | 89% | Partant |

Table 8 – Exemples de prédictions de l'interface

## 6.6 Logs et monitoring

L'application affiche des logs détaillés dans la console :

- Données reçues du formulaire
- Encodage des variables catégorielles
- Vérification du vecteur final (NaN, shape)
- Résultat de la prédiction (probabilité, niveau)

## 6.7 Utilisation de l'API REST

```
# Exemple d'appel API avec curl
curl -X POST http://127.0.0.1:5000/api/predict \
  -H "Content-Type: application/json" \
  -d '{
    "recency": 30,
    "frequency": 5,
    "monetary": 500,
    "age": 35,
    "satisfaction": 4,
    "support_tickets": 0,
    "favorite_season": "Hiver",
    "spending_category": "Medium",
    "account_status": "Active"
  }'
```

# 7 Conclusion et Perspectives

## 7.1 Résumé des réalisations

**Bilan du projet**

- **Exploration** : analyse complète de 52 features, identification des anomalies
- **Préparation** : nettoyage, feature engineering, suppression leakage (9 features)
- **Modélisation** : 4 modèles testés, Gradient Boosting retenu (F1=0.83, AUC=0.947)
- **Segmentation** : 4 clusters clients + outliers, recommandations actionnables
- **Déploiement** : application Flask fonctionnelle avec API REST

## 7.2 Points forts du projet

1. **Rigueur scientifique** : split avant imputation, pas de data leakage
2. **Modèle performant** : F1-Score de 0.83, AUC de 0.947
3. **Segmentation exploitable** : 4 clusters aux profils distincts
4. **Interface professionnelle** : jauge de risque, scores RFM, historique
5. **Code versionné** : GitHub avec commits réguliers

## 7.3 Limites identifiées

- **FavoriteSeason domine le modèle** (56.5% de l'importance) → possible biais temporel
- **23.4% d'outliers** retirés de la segmentation → perte d'information
- **Donées datées** : période 2009-2011, centrées sur le Royaume-Uni
- **Age imputé par médiane** : 30% de valeurs manquantes
- **KNN moins performant** sur données tabulaires complexes (F1=0.818)

## 7.4 Perspectives d'amélioration

- Test de modèles deep learning (MLP, TabNet)
- Ajout de données plus récentes (2020-2025)
- Intégration de SHAP pour l'interprétabilité
- Mise en production sur cloud (AWS, GCP, Heroku)
- Dashboard temps réel avec Plotly Dash
- A/B testing des recommandations marketing
- Système de monitoring du modèle (drift detection)
- Ajout de plus de features externes (météo, économie)

## 7.5 Livrables du projet

- **Code source** : dépôt GitHub complet
- **Application Flask** : interface de prédiction fonctionnelle
- **Modèle entraîné** : best_model_churn.pkl
- **Notebooks** : analyses, modélisation, segmentation
- **Rapport** : présent document (LaTeX)
- **Présentation** : slides pour la soutenance

# Annexes

## A. Structure complète du projet

```
ProjetML-1/
 app/
    app.py                  # Application Flask
    templates/
        index.html          # Interface utilisateur
 models/
    best_model_churn.pkl    # Modèle Gradient Boosting
    scaler.pkl              # StandardScaler
    imputer.pkl             # SimpleImputer
    feature_names.pkl       # Noms des 103 features
    ...                     # Autres modèles (kmeans, pca)
 notebooks/
    test.py                 # Script principal
    ...                     # Notebooks d'analyse
 data/
    raw/                    # Données brutes
    processed/              # Données nettoyées
    train_test/             # Split train/test
 reports/
    segmentation_dashboard.png
    arbre_decision.png
    segmentation_recommandations.csv
 src/
    preprocessing.py
    train_model.py
    utils.py
 requirements.txt
 README.md
 .gitignore
```

## B. Environnement technique

| Catégorie | Technologies |
|---|---|
| Langage | Python 3.10+ |
| Data | Pandas, NumPy |
| Visualisation | Matplotlib, Seaborn |
| Machine Learning | Scikit-learn (GB, RF, KNN, DecisionTree) |
| Déploiement | Flask, HTML/CSS |
| Versioning | Git, GitHub |

## C. Installation et exécution

```
# Cloner le repository
git clone https://github.com/myriamBenAbd0607/ProjetML.git
cd ProjetML

# Cr er l'environnement virtuel
python -m venv venv
```

```
source venv/bin/activate  # Linux/Mac
venv\Scripts\activate     # Windows

# Installer les d pendances
pip install -r requirements.txt

# Lancer l'application Flask
cd app
python app.py

# Ouvrir dans le navigateur
# http://127.0.0.1:5000
```

## D. Références


— Documentation Scikit-learn : `https://scikit-learn.org`

— Documentation Flask : `https://flask.palletsprojects.com`

— Documentation Pandas : `https://pandas.pydata.org`

— Support de cours : Module Machine Learning - GI2